

Emergent Complexity

Initial Assignment

George Tsoukalas

September 2026

Overview

Hi! You will build a small interactive cellular-automaton laboratory and use it to explore how complex behavior can emerge from simple rules.

There is no fixed due date for this assignment. Work through it at your own pace, and check in with me as questions arise. You are welcome to ask about programming, experiment design, cellular automata, or anything else related to the project.

You are encouraged to (and should!) use AI throughout the project, especially for implementation, debugging, and learning unfamiliar concepts. For example, you could use Codex through ChatGPT, Claude Code, or Gemini/Antigravity, but the choice is up to you. I believe UT Austin gives free AI support via ChatGPT Edu if you don't have one already!

You are still responsible for understanding your code, testing that it works, and being able to explain the important design decisions. In your report, describe how you used AI and which parts of the project it helped with.

Your final artifact will include a runnable website, a GitHub repository, and a report describing your experiments and findings. If you need some help setting up technical stuff, like figuring out how to use github or make a website - please ask! (But one of the points of AI is that it can help you :))

1 Watch and Explore the Game of Life

Begin by watching Emergent Garden's video on artificial life:

<https://www.youtube.com/watch?v=2g-CrQfYNtE>

It's a pretty cool and high quality channel. As you watch, consider what makes a system appear life-like. Think about emergence, self-organization, reproduction, variation, and selection.

You will begin with Conway's Game of Life. Life is a two-dimensional, binary-state cellular automaton. Each cell is either on or off, and each cell has eight neighbors in its *Moore neighborhood*. The *Moore Neighborhood* is the immediate neighbors of a cell. In 2D - there are 8 neighbors, in 3D, 26.

The standard rule is written

B3/S23

This means that an off cell becomes on if exactly three neighbors are on, while an on cell survives if it has exactly two or three on neighbors.

Task 1: Build and explore

Choose a programming language and build an interactive Game of Life website. Host the code in a github repository and host the site! You should be able to do this for free. It should allow users to draw cells, run and pause the simulation, step forward, reset the grid, and adjust the speed.

Once your simulator works, run many simulations using randomly generated initial configurations. Try different initial densities. Look carefully at what happens over time.

Identify any interesting patterns, recurring structures, moving objects, clusters, boundaries, or other substructures that appear. You do not need to know the names of these structures in advance. Include several examples in your report and describe what you observed.

2 Make the Rule Editable

For an **outer-totalistic** rule, the next state depends on whether the center cell is on or off and on the *number* of on neighbors, from zero through eight.

Such rules can be written as

`B[ birth counts ]/S[ survival counts ].`

For example, B3/S23 is Conway's Game of Life and B36/S23 is HighLife. There are

$$2^{18} = 262,144$$

possible binary outer-totalistic rules in this neighborhood.

Task 2: Generalize the simulator

Modify your website so that users can change the birth and survival conditions. Make the mechanism for changing the rules visual and intuitive. Make it easy to see which neighbor counts cause birth and survival. Explore in the HighLife rule and catalogue some patterns that you find.

3 Explore a Small Rule Space

Next, let's try to understand what happens among randomly sampled rules, since it's a pretty big task to do all 2^{18} of them. Randomly sample 100 outer-totalistic rules and run each one from a small number of random initial configurations. Look for broad differences in behavior, such as extinction, stability, oscillation, growth, disorder, mobility, or possible self-reproduction.

A rule that produces no interesting behavior in one experiment may still produce interesting behavior from another initial condition, so do not treat a single simulation as definitive.

Task 3: Compare rules

Classify the sampled rules using categories that make sense to you. Support your observations with a few measurements, images, or animations. Choose at least one particularly interesting rule for closer investigation.

If your sample does not contain an interesting rule, try *designing* some rules and explain how you would search for more interesting behavior.

4 Choose One Open-Ended Investigation

Choose one of the following extensions:

Option A: Three-dimensional cellular automata

Implement a three-dimensional cellular automaton simulator on the website with an understandable and cool visualization. Try out various rule-sets, and explore the structures appear and compare them with two-dimensional systems. How big can you make the 3D grid on your computer? Investigate statistical behavior of random rules, with many random initial configurations. Report back what you find.

Option B: Noise and perturbations

Introduce random perturbations into the simulation 2D cellular automata simulations. For example, randomly turn cells on or off with a small probability at each step. Try different amounts of noise and see how it changes. For the rules you explored in the previous task, study which patterns are robust and which disappear under the presence of various levels of noise. Report back what you find. Can you find any evidence of thresholding behavior?

Option C: Multiple cell states

Extend the system so that cells can have more than two possible states. Make it look good on the website! Choose rules for how the states change and investigate what new kinds of patterns or structures appear. Theorize some possible design principles for making interesting rules, and see if they hold! Build understanding of what rules could lead to interesting behavior, and design some! Report on this investigation, and what interesting structures you find.

There will also be technical implementation questions and issues in implementing these. Make sure to discuss these technical and design choices you made to carry this part out - you have freedom here to try various things!

5 Final Report and Submission

Write a report (in docs, overleaf, or something else - your choice!) explaining:

1. what you built and how it works;
2. what questions you investigated;
3. which rules and initial conditions you used;

4. what patterns or behaviors you observed;
5. how you used AI;
6. what limitations you might have encountered; and
7. what general thoughts you had about this work - did anything excite you? What did you find interesting? What would you like to learn more about?

Include links or screenshots so that someone else can run your website and reproduce at least some of your observations. When you are done, send me an email with a link to your GitHub repository, your website, and your report. We'll go over the websites in a chill, low-stakes, group presentation in our first meeting.

If you aren't able to find time to do this - no worries! It probably means that this project would be hard to commit time to, and that's fine!